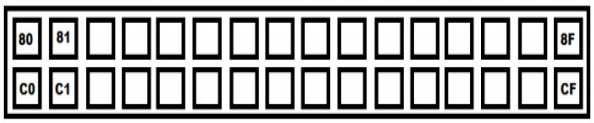
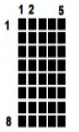
EMT 3202: Mechatronic Systems Programming II - Notes

Lesson 1: a

Introduction:

LCDs (Liquid Crystal Displays) are used for displaying status or parameters in embedded systems. There are various configurations of LCD displays but a common one is the LCD 16x2. The LCD 16x2 has 2 lines with 16 characters in each line. Each character is made up of 5x8 (column x row) pixel matrix.

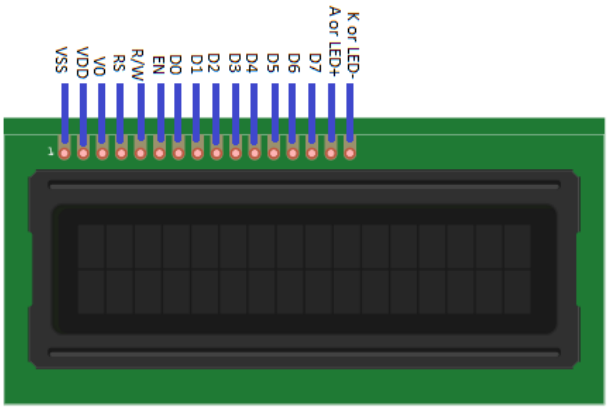




5x8 sized pixel

The positioning of the elements is as shown above. The LCD consists of 2 rows. The 1st starts from address 0x80 to 0x8F while the 2nd row starts from address 0xC0 to 0xCF. As shown each character is made from a row of 40 pixels arranged in a 5x8 matrix. The character usually utilizes 7 rows with the last row reserved for the cursor.

LCD 16x2 is a 16 pin devices which has 8 data pins (D0-D7) and 3 control pins (RS, RW, EN). The remaining 5 pins are for power supply and backlight for the LCD. The 3 control pins help us configure the LCD in command mode or data mode. They also help to configure the LCD into read mode or write mode. LCD 16x2 can be used in 4-bit mode or 8-bit mode depending on requirement of application. In order to use it we need to send certain commands to the LCD in command mode and once the LCD is configured according to our need, we can send the required data in read mode.



No.	PIN	Function
1	VSS	Ground
2	VCC	+5V
3	VEE	Contrast control
4	RS	Register Select
5	RW	Read/Write pin
6	EN	Enable pin (H-L pulse)
7-14	D0-D7	Data pins for data transfer
15	LED+	+5V
16	LED-	Ground

Pin Description:

PIN 1 (VSS) and 2 (VCC) are used to supply power to the LCD i.e. ground and VCC.

Pin 3 - VEE pin; this pin is used for adjusting the contrast of the display. Voltage on this pin defines contrast on display, lower the voltage, higher the contrast. We can connect 4.7 k pot for contrast adjustment or simply connect this pin to ground to get maximum contrast.

Pin 4 –RS: Register Select pin:
RS = 0: Data on the D0 to D7 pins is considered as a command.
RS = 1: Data on the D0 to D7 pins is considered as data to display on LCD16x2.

Pin 5 – RW: Read / Write pin
RW = 0: Write data to the LCD
RW = 1: Read data from the LCD

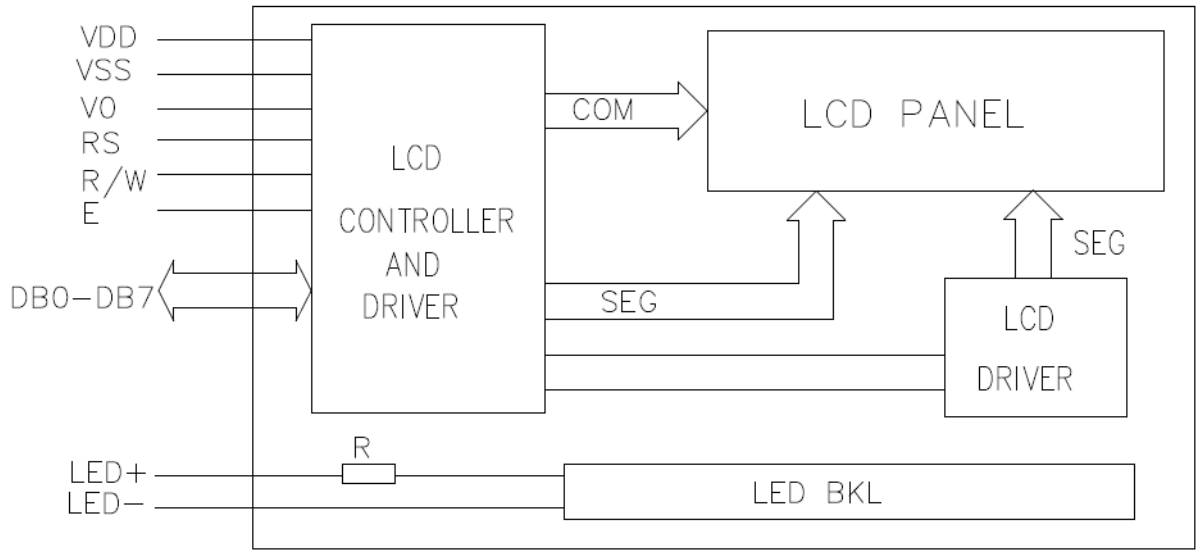
Pin 6 –E: Enable; this pin is used to latch the data present on the data pins D0 to D7. High to low pulse with a minimum width of 450 ns is required to latch the data to the display.

Pins 7:14 - DATA pins D0 to D7; data pins are used to send data/command to the LCD16x2 as parallel 8 data bits.

Pin 15:16 - LED + and LED -; liquid Crystal Displays don't have their own light like seven segment displays. Therefore, the module has a backlight LED. Supply to this LED is provided through these pins.

Operation:

The internal configuration of components in an LCD 16x2 is as shown in the figure below:



A key component LCD control circuit is the LCD controller and driver. This is an IC programmed to light the different sections of each digit according the commands from the microcontroller. A common controller IC is the HD44780U from Hitachi. Thus in order to properly understand how to operate the LCD, we need to check the HD44780U datasheet.

Command Summary:

From the data sheet analysis we can observe that the following codes are used when interfacing with a typical HD44780U controller LCD. The following link also offers an analysis of the data sheet and shows the setting of the various bits to achieve desired functions: https://www.mil.ufl.edu/3744/docs/lcdmanual/commands.html

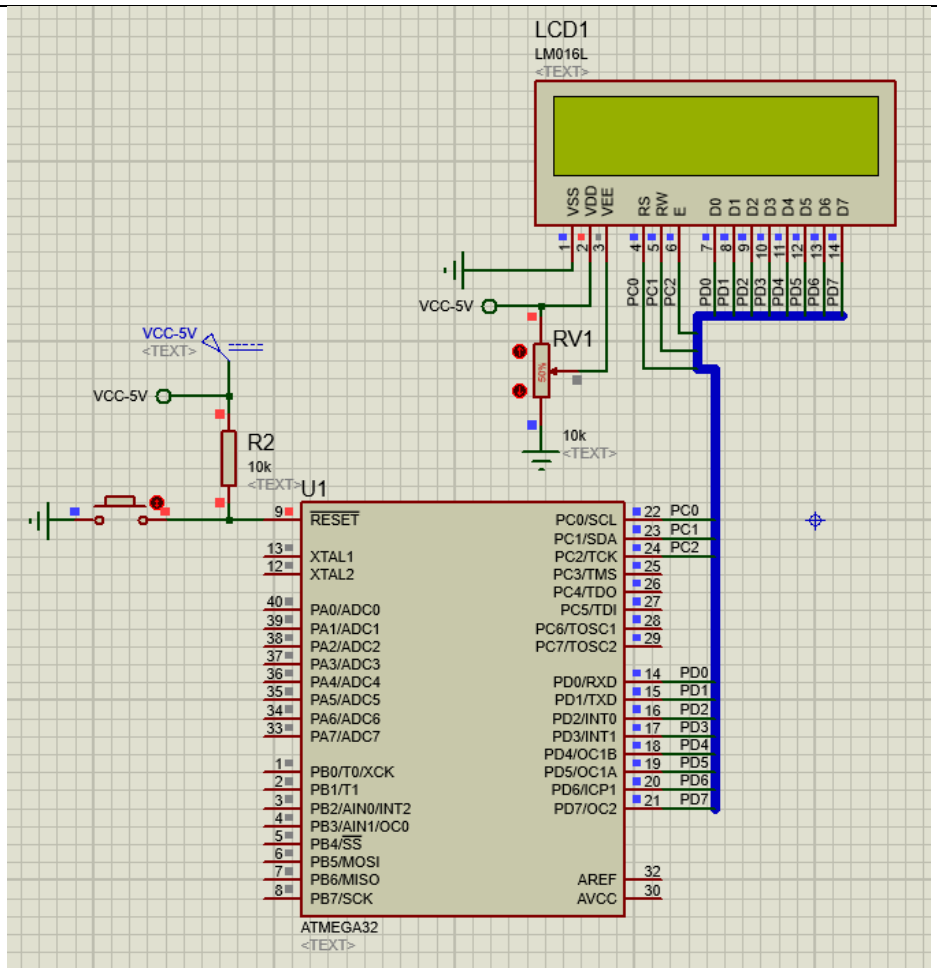
Code (Hex)	Command to LCD	Execution Time
0x01	Clear the display screen	1.64 ms
0x02	Return display to its home position	40 us
0x04	Shift the cursor left (e.g. data gets written from right to left)	40 us
0x06	Shift the cursor right (e.g. data gets written form left to right)	40 us
0x05	Shift display right	40 us
0x07	Shift display left	40 us
0x08	Display OFF, cursor OFF	40 us
0x0A	Display OFF, cursor ON	40 us
0x0C	Display ON, cursor OFF	40 us
0x0E	Display ON, cursor blinking	40 us
0x0F	Display ON, cursor blinking	40 us
0x10	Shift cursor position to the left	40 us
0x14	Shift cursor position to the right	40 us
0x18	Shift entire display to the left	40 us
0x1C	Shift entire display to the right	40 us
0x80	Place the cursor to the beginning of the 1st row	40 us
0xC0	Place the cursor to the beginning of the 2nd row	40 us
0x38	2 lines, 5x8 matrix, 8-bit mode	40 us
0x28	2 lines, 5x8 matrix, 4-bit mode	40 us
0x30	1 line, 8-bit mode	40 us
0x20	1 line, 4-bit mode	40 us

Circuit and Programming:

A typical LCD microcontroller connection is as shown in figure 1. After designing the circuit in Proteus, the operating code can now be written in Atmel studio. From the data sheet we observe that there are 2 main interactions between the microcontroller and the LCD driver IC i.e. writing Cycle: were we send commands and data to the IC and Reading Cycle: Where we receive data from the IC. We will mainly be writing commands and data so those are the function we write first.

```
/*LCD command write function*/
void LCD_Cmd(unsigned char cmd){
        LCD_Data_Port = cmd;
        LCD_Cmd_Port &= ~(1<<RS); /* RS=0 command reg. */
        LCD_Cmd_Port &= ~(1<<RW); /* RW=0 Write operation */
        _delay_us(1);
        LCD_Cmd_Port |= (1<<EN);  /* Enable pulse ON */
        _delay_us(1);
        LCD_Cmd_Port &= ~(1<<EN); /* Enable pulse OFF */
        _delay_ms(2);
}
```

```
/*LCD data write function */
void LCD_Char (unsigned char char_data){
        LCD_Data_Port = char_data;
        LCD_Cmd_Port |= (1<<RS);  /* RS=1 Data reg. */
        LCD_Cmd_Port &= ~(1<<RW); /* RW=0 write operation */
        _delay_us(1);
        LCD_Cmd_Port |= (1<<EN);  /* Enable Pulse ON */
        _delay_us(1);
        LCD_Cmd_Port &= ~(1<<EN); /* Enable Pulse ON */
        _delay_ms(2);
}
```



With these two command write and data write functions we can implement other important functions i.e. LCD initialization, LCD clear and writing a string of characters to the LCD.

```
/*LCD Initialize function */
void LCD_Init (void){
       LCD_Cmd_Dir = 0xFF;   /* Make LCD command port direction as output pins*/
       LCD_Data_Dir = 0xFF; /* Make LCD data port direction as output pins*/
       _delay_ms(50);               /* LCD Power ON delay always >15ms */
       LCD_Cmd(0x02);              /* Return display to its home position */
       LCD_Cmd(0x38);              /* Initialization of 16X2 LCD in 8bit mode */
       LCD_Cmd(0x0F);              /* Display ON Cursor Blinking */
       LCD_Cmd(0x06);              /* Auto Increment cursor */
       LCD_Cmd(0x01);              /* Clear display */
}
```

```
/*Clear LCD Function*/
void LCD_Clear(void){
        LCD_Cmd(0x01);    /* clear display */
        LCD_Cmd(0x02);    /* Return display to its home position */
}
/*Send string to LCD function */
void LCD_String (char *str){
        int i;
        /* Send each char of string till the NULL */
        for(i=0;str[i]!=0;i++){
                LCD_Char(str[i]);
        }
}
```

Lastly we can implement a more convenient function for writing a string of characters to the LCD which we are able to specify the position where to display the characters.

```
/*Send string to LCD with xy position */
void LCD_String_xy (char row, char pos, char *str){
        if (row == 0 && pos<16){
                LCD_Cmd((pos & 0x0F)|0x80);/* Command of first row and required
position<16 */
        }
        else if (row == 1 && pos<16){
                LCD_Cmd((pos & 0x0F)|0xC0);/* Command of second row and required
position<16 */
        }
        LCD_String(str);                  /* Call LCD string function */
}
```

Thus with these 6 key functions we can now write our main function. First we write a simple program to test the functionality of our code.

```
/*Typical Header file inclusion and CPU frequency definition*/
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>

/*Function Declarations*/
void LCD_Cmd(unsigned char cmd);
void LCD_Char(unsigned char char_data);
void LCD_Init(void);
void LCD_Clear(void);
void LCD_String(char *str);
void LCD_String_xy(char row, char pos, char *str);

/*Our main program*/
int main(void)
{
        LCD_Init();               /* Initialize LCD */
        LCD_String("MSP II:EMT 3202");    /* Write a string on 1st line of LCD*/
        LCD_Cmd(0xC0);                    /* Go to 2nd line*/
        LCD_String("Hello World");        /* Write string on 2nd line*/
}
```

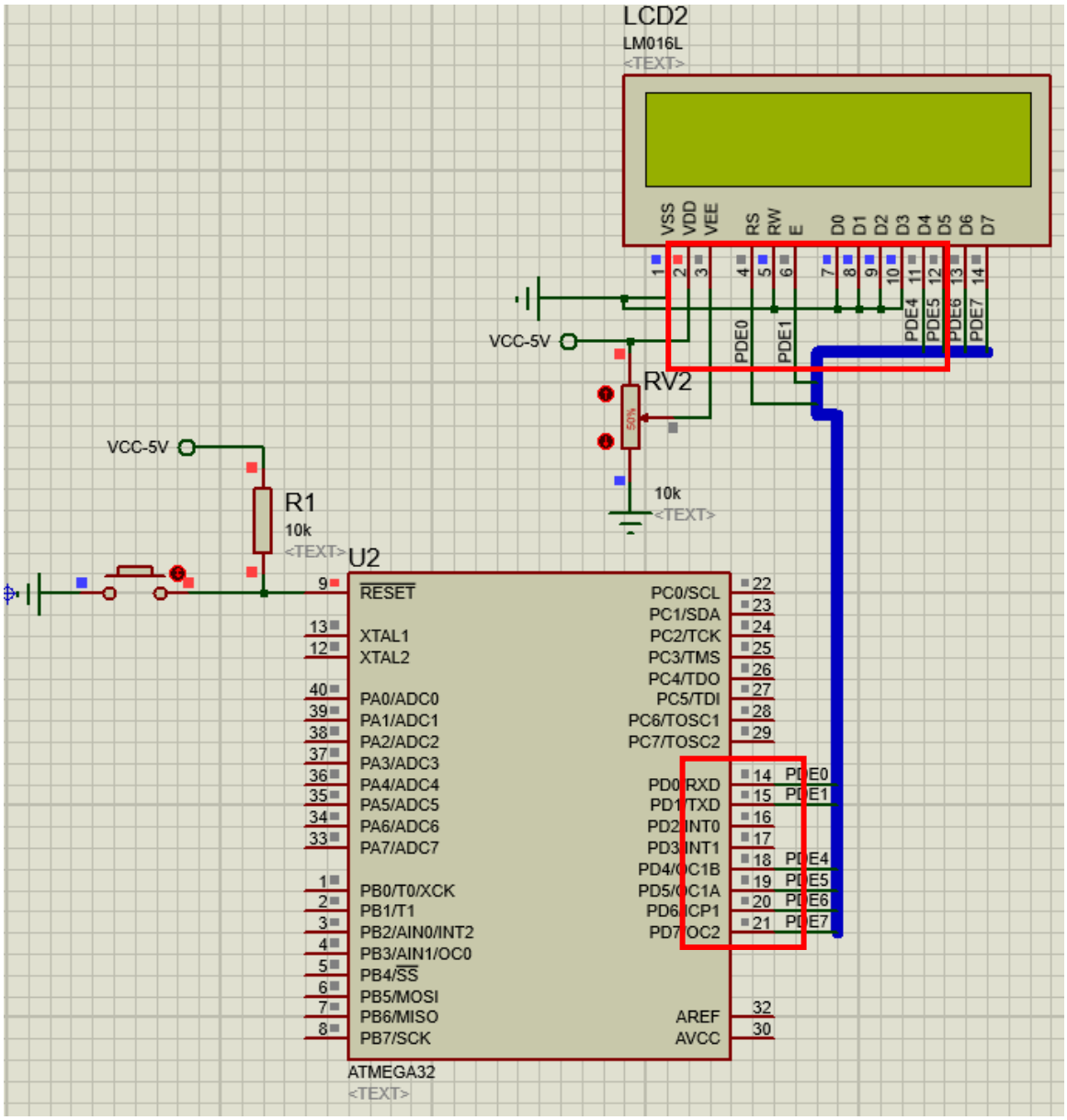
NB: Since this and other subsequent projects will have large sections of code, it is preferred to separate our code files e.g. a file containing the LCD functions and leave our main.c file with just the main function. This will make the organization and debugging of our code easier.

Class activity. Convert the above code into a separate LCD function file and include a header file.

8 bit mode vs 4 bit mode

In many engineering application, it is always desirable to reduce the number of MCU pins used to operate a certain device. In the above LCD example, we use 11 MCU pins to control the LCD i.e. 3 control pins (RS, RW and E) and 8 data pins (D0 – D7). The LCD driver IC has the capacity to be operated using 4 instead of 8 data pins and ground the RW pin. This enables us to save 5 pins for other uses.

When operating the LCD using all data pins, it is known as 8 bit mode and when using 4 pins it is 4 bit mode. To implement this mode operation the circuit arrangement will change i.e. disconnection of D0-D3 of the LCD and the LCD command functions will also change. These are the 2 major changes needed to operate in 4 bit mode. While operating under 4 bit mode, we send the 8 bits of data, 4 bits at a time i.e. 4 Higher bits and then 4 Lower bits. The new LCD circuit is as shown in the figure below with the modifications to the pin layout as highlighted in red.



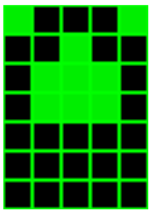
Next is the implementation of the new write functions. Since read function is rarely used, we ground the RW pin to keep it constantly in the Write cycle mode. The changes to the code mainly occur in the main functions, i.e. write command and write data as well as the Initialize function. These new function definition are as shown below:

```c
/*Useful pin and port definitions*/
#define LCD_Dir DDRD
#define LCD_Port PORTD

#define RS PD0
#define EN PD1
/*LCD command write function*/
void LCD_Cmd(unsigned char cmd){
        /*Sending the first nibble of data (Higher 4 bits)*/
        LCD_Port = (LCD_Port & 0x0F) | (cmd & 0xF0);/* Sending upper nibble */
        LCD_Port &= ~ (1<<RS);            /* RS=0, command reg. */
        LCD_Port |= (1<<EN);              /* Enable pulse ON */
        _delay_us(1);
        LCD_Port &= ~ (1<<EN);            /* Enable pulse OFF */
        _delay_us(200);
        /*Sending the second nibble of data (Lower 4 bits)*/
        LCD_Port = (LCD_Port & 0x0F) | (cmd << 4);/* Sending lower nibble */
        LCD_Port |= (1<<EN);              /* Enable pulse ON */
        _delay_us(1);
        LCD_Port &= ~ (1<<EN);            /* Enable pulse OFF */
        _delay_ms(2);
}
/*LCD data write function */
void LCD_Char (unsigned char char_data){
        /*Sending the first nibble of data (Higher 4 bits)*/
        LCD_Port = (LCD_Port & 0x0F) | (char_data & 0xF0);/* Sending upper nibble */
        LCD_Port |= (1<<RS);              /* RS=1, data reg. */
        LCD_Port |= (1<<EN);              /* Enable pulse ON */
        _delay_us(1);
        LCD_Port &= ~ (1<<EN);            /* Enable pulse OFF */
        _delay_us(200);
        /*Sending the second nibble of data (Lower 4 bits)*/
        LCD_Port = (LCD_Port & 0x0F) | (char_data << 4);  /* Sending lower nibble */
        LCD_Port |= (1<<EN);              /* Enable pulse ON */
        _delay_us(1);
        LCD_Port &= ~ (1<<EN);            /* Enable pulse OFF */
        _delay_ms(2);
}
/*LCD Initialize function */
void LCD_Init (void){
        LCD_Dir = 0xFF;  /* Make LCD command port direction as output pins*/
        _delay_ms(20);   /* LCD Power ON delay always > 15ms */

        LCD_Cmd(0x02);   /* Return display to its home position */
        LCD_Cmd(0x28);   /* 2 line 4bit mode */
        LCD_Cmd(0x0C);   /* Display ON Cursor OFF */
        LCD_Cmd(0x06);   /* Auto Increment cursor */
        LCD_Cmd(0x01);   /* Clear display */
}
```

Custom Characters

In various scenarios during embedded system programming, it is required to use custom characters in order to develop user friendly interfaces that display relevant information and functionalities as shown in the figure below. The LCD driver has a character graphics RAM (CGRAM), where custom icon data can be stored and displayed. Before inputting the custom character data into the CGRAM we need to design the icon pixel data.

5x8 matrix (bottom row is for the cusor)



bin:0b01110 hex:0x0E
bin:0b11011 hex:0x1B
bin:0b10001 hex:0x11
bin:0b10001 hex:0x11
bin:0b11111 hex:0x1F
bin:0b11111 hex:0x1F
bin:0b11111 hex:0x1F

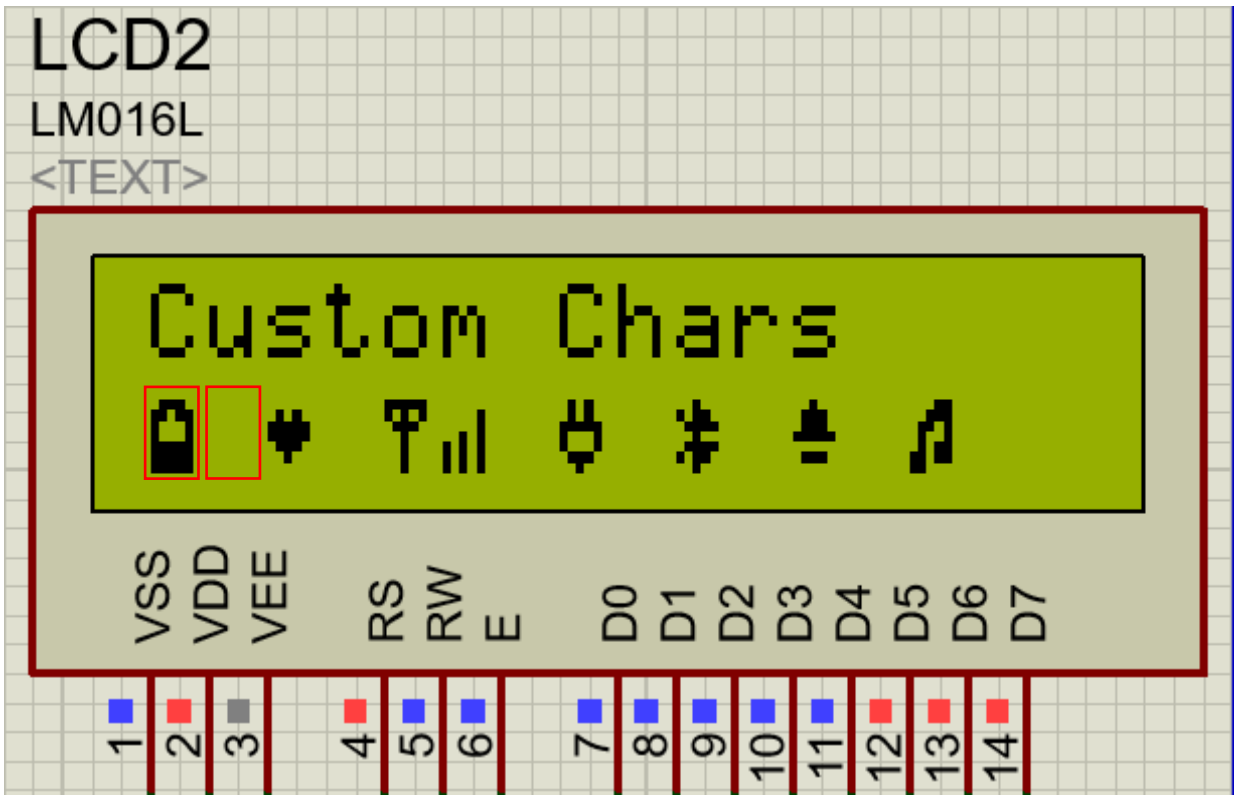
As mentioned, each character is composed by lighting a 5x8 matrix of pixels. As shown above. To display a half filled battery icon, the pixel data is as shown below. In the image, only 7 rows of the 8 row matrix is used with the 8th row reserved for use by the cursor. An ON pixel is represented by a 1 and OFF by 0. From the figure we can see that each row of pixels is converted to binary and then hexadecimal digits. These 7 hexadecimal digits are then loaded into the CGRAM and the icon can then be stored in the LCD IC and later used within the main program. There are various online tools used to generate custom (5x8) LCD characters e.g. https://www.quinapalus.com/hd44780udg.html.

```
/*Typical Header file inclusion and CPU frequency definition*/
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include "LCD_Code.h" /*Inclusion of our own defined header file*/

/*Array to store hexadecimal values for custom characters */
const short Custorm_Chars5X8[] = {
        0x0E,0x1B,0x11,0x11,0x1F,0x1F,0x1F,0x00,// Code for CGRAM memory space 0:
Battery
}

/*Function Definition: To load the hex data into the first space in CGRAM*/
void Load_Custom_Char(){
        /*Load custom char into the CGROM*/
        LCD_Cmd(0x40);    // Set CGRAM Address
        for (int i = 0; i <= sizeof(Custorm_Chars5X8); i++)
        LCD_Char(Custorm_Chars5X8[i]);
        LCD_Cmd(0x80);   // Return to Home
}


/*Our main program*/
int main(void){
        LCD_Init();                /* Initialize LCD */
        Load_Custom_Char();        /* Load custom characters into CGRAM */
        LCD_Clear();               /* Clear LCD */
        LCD_String("Custom Chars");/* Display string on screen */
        _delay_ms(1000);
        LCD_Cmd(0xC0);             /* Place cursor on the second row */
        LCD_Char(0x00);            /* Display the character in address 0x00 of CGRAM*/

        return 0;
}
```

Individual Exercise (10 marks)

1. Create the message shown on the picture below using the 16x2 LCD: It consists of the word "Custom Chars" on the 1st row. The 2nd row: Battery half-filled, space, heart, space, signal sign, signal strength, space, power plug, space, Bluetooth, space, bell icon, space, music icon.
2. When the system is turned on, the message below should display for 3 seconds and then the message should scroll to the left. It should move by 1 element space until all characters on the screen are no longer visible.

- *You will demonstrate and explain the working program to the lecture with a 5 minutes question and answer session. Marks will be awarded for a compact program, clear explanation and deep understanding of the concept.*
- *This exercise is part of the Continuous Assessment of this course.*
- *Failure to submit on the appointed time will result in a zero score for the exercise.*



Lesson 2:

4x4 Keypad Interfacing:

In various embedded system application, the keypad is a useful way of obtaining input information from a user e.g. ATMs, vending machines, security locks, telephones, card readers, control panels etc. In many daily dealings, one tends to interact with keypads quite often thus it is important to learn how to interface microcontrollers with it.

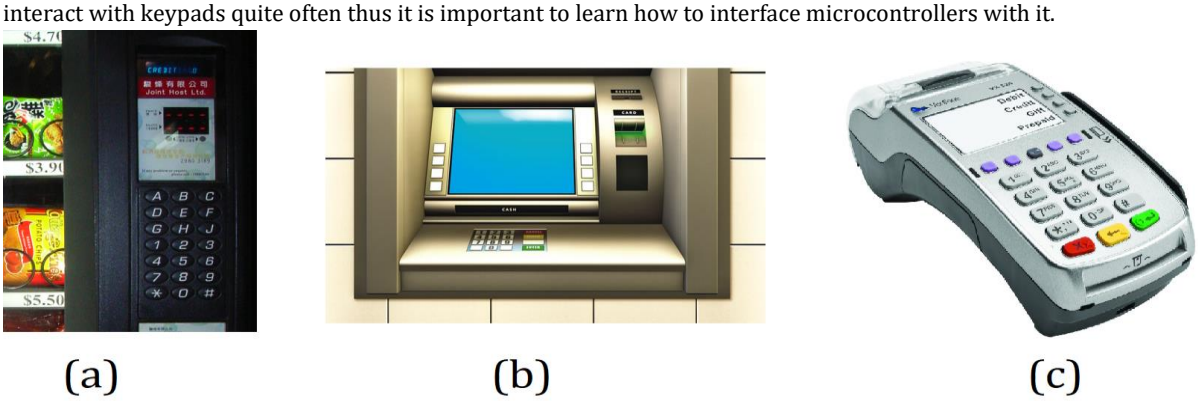
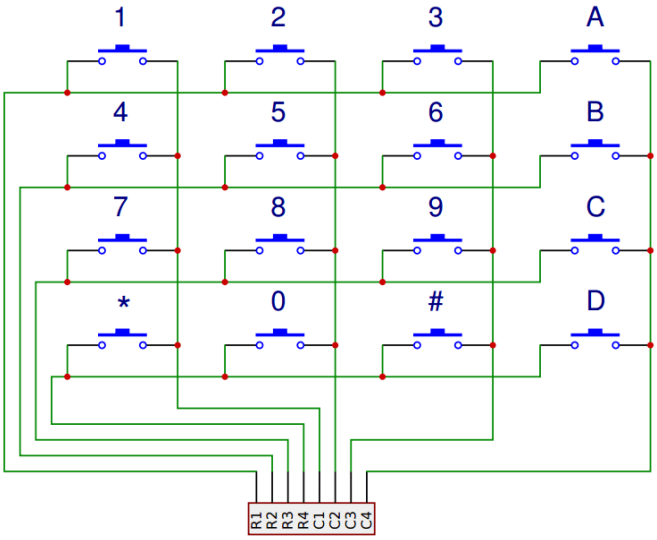


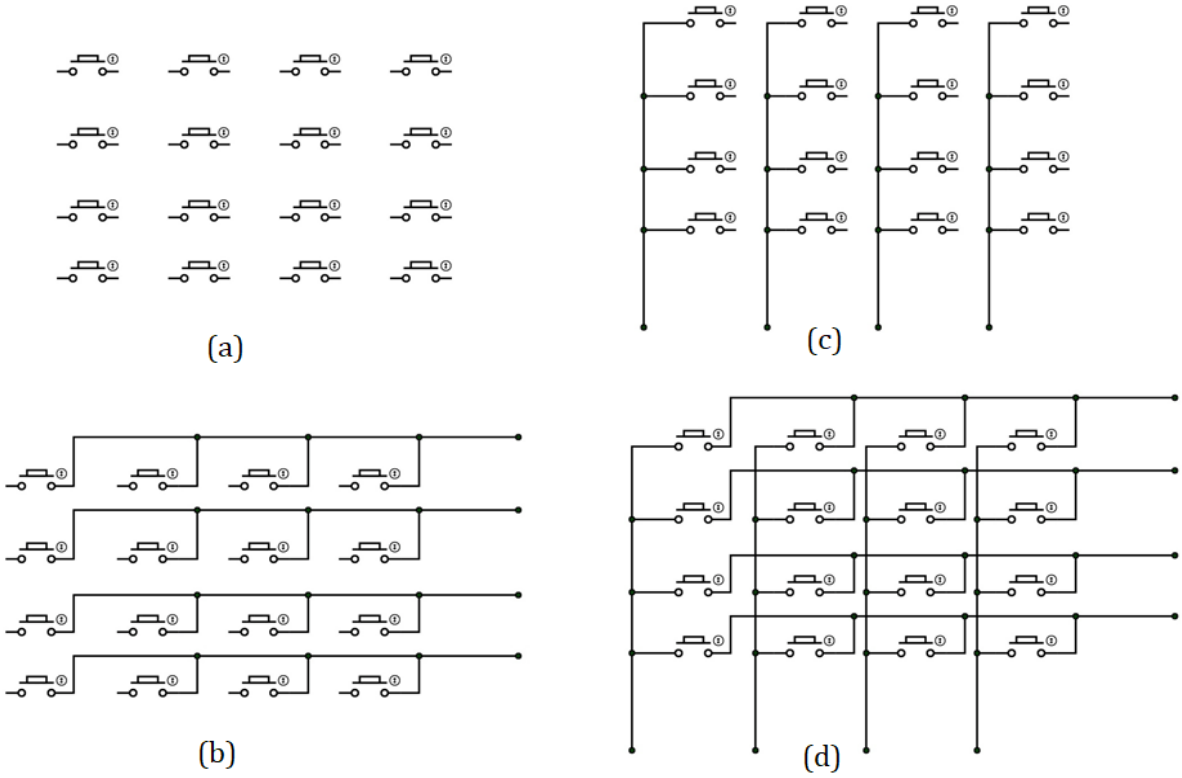
Figure 1: (a) Vending machine, (b) ATM machine and (c) Credit card reader

Keypad structure:



When we want to interface one key to the microcontroller then it needs one GPIO pin but when we want to interface many keys like 9, 12 or 16 etc., then it may acquire all GPIO pins of microcontroller. To save some GPIO pins of microcontroller, we can use matrix keypad. Matrix keypad is nothing but keys arrange in row and column i.e. if we want to interface 16 keys to the microcontroller then we require 16 GPIO pins but if we use matrix 4x4 keypad then we require only 8 GPIO pins of microcontroller.

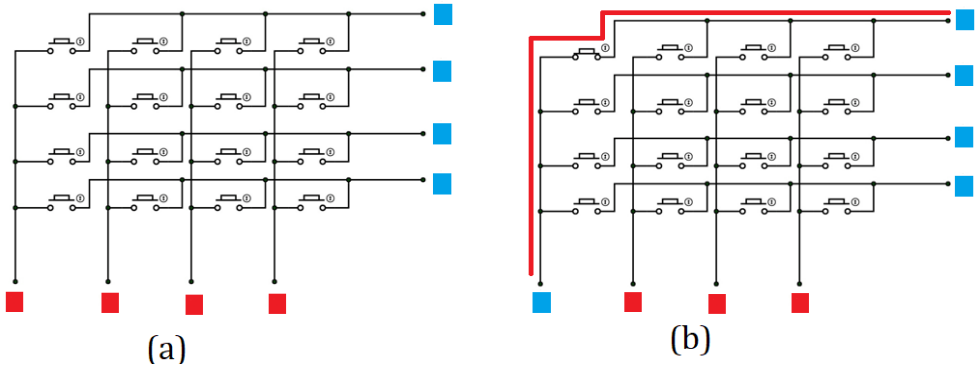
Keypads come in various shapes and sizes but the most common configurations are the 3x4 and 4x4 keypads. These contain 12 and 16 buttons respectively which the user can interact with. The general inner structure of these keypads is as shown in the above right figure. The keypad consists of a set of push buttons arranged in a matrix structure. The buttons are connected on one side by a wire that connects all the buttons on the same row and the other side the wire that connects the button to the rest of the buttons on the same column. This is shown in the figure below.



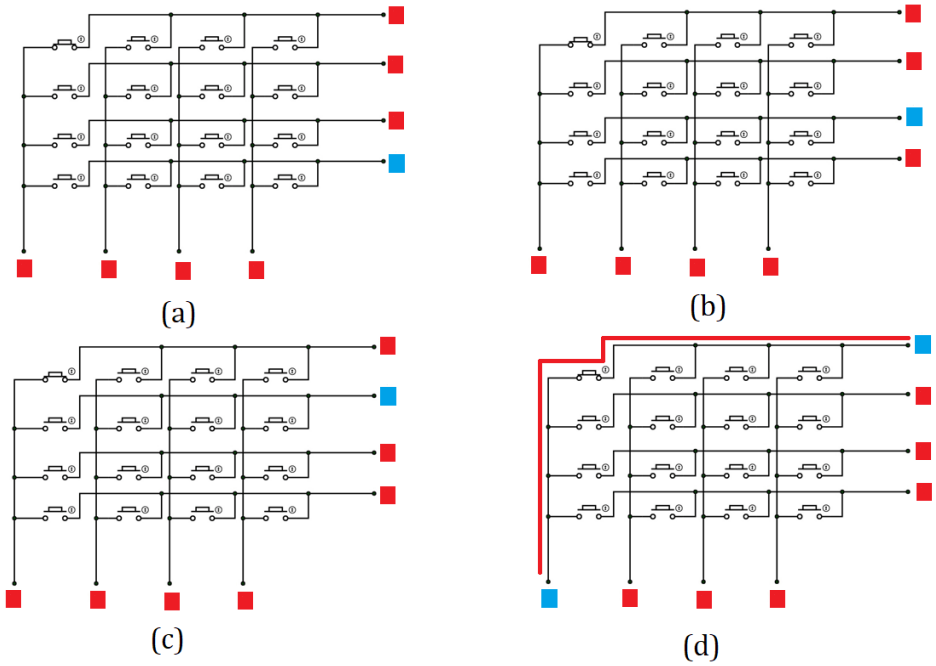
This structure enables us to uniquely identify which button is pressed by energizing the rows and checking which column lights up and once this is known, we energize the columns and check the rows to see which one is ground.

Keypad matrix operation: Scanning of Keys

There are various ways of scanning for pressed keys. One of the reliable ways is shown below. First the Rows are set as OUTPUTS while the Columns are set as INPUTS. Next we Ground the Rows i.e. set them LOW (=0000) and set the Columns as HIGH as shown in figure (??) (a). If the top left button is pressed, the column will be grounded and if we look at the column values we can find which column the button belongs to.

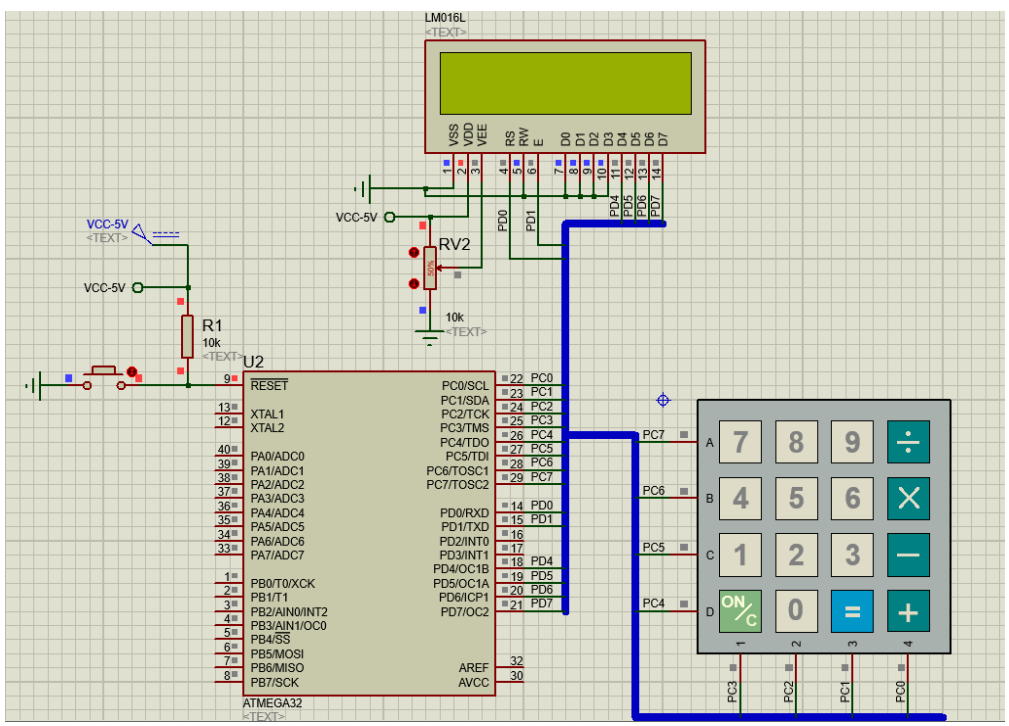


Once we know the column, we then proceed to check for the row. This is done as shown in figure (??). As shown, each Row pin is set LOW progressively while observing the column values. Rows = {0111, 1011, 1101, 1110}. When we ground the right Row, we will obtain a grounding of one of the same column pin and thus we can identify the row which the button belongs to i.e. Row = 1110, the column will be = 0111. In every other case the column is 1111.



With both the column number and the row number, we can identify which button was pressed.

Circuit and Code Implementation:



The above figure shows the Proteus circuit diagram that is used to interface the microcontroller with the 4x4 keypad. The LCD screen from previous lessons is also connected to act as an output device and to check the key that has been pressed. PORTD is used to control the LCD while PORTC is used to interface with the keypad. Firstly we implement the first section

of code that is able to look at the column that has been pressed. Initially we create an array that holds the layout of the keypad and the character each button represent so that we can display the right character on the LCD.

Next a function is created that will be able to set the INPUT and OUTPUT pins and check for any button press on the column side. This is done using 2 do-while loops which also provides some button debouncing and prevents false readings. Once we know the column of the pressed button, we look for the appropriate row. This is done by cycling through all the row pins, and ground them one at a time. After this step, we use the column and row values to return the button character that has been pressed as the return value of our function.

```
/* Keypad array holding the keys in a grid arrangement*/
unsigned char keypad[4][4] = {        {'7','8','9','/'},
                                      {'4','5','6','*'},
                                      {'1','2','3','-'},
                                      {' ','0','=','+'}};

/* Function that checks the key that has been pressed on the keypad */
char check_Keypad()
{
          while(1)
          {
                    KEY_DDR = 0xF0;     // Set the rows as outputs and the columns as inputs
                    KEY_PORT = 0x0F;    // Set the columns HIGH and the rows LOW.

/*With the columns HIGH and the rows LOW, thus the KEY_PIN == 0x0F(0b0000 1111).
  If a button is pressed, the value of the column pin will be grounded and
  KEY_PIN will change from 0x0F to some other number 0x0E i.e.
  0x0E(0b0000 1110) i.e. a button in the 4th column was pressed.
*/
                    do
                    {
                              do
                              {
                                        _delay_ms(20);                // 20ms key debounce time
                                        colNum = (KEY_PIN & 0x0F);    // read status of column
                              }while(colNum == 0x0F);                 // check for any key press

                                        _delay_ms (40);               // 20 ms key debounce time
                                        colNum = (KEY_PIN & 0x0F);
                    }while(colNum == 0x0F);

/*Once we have established which column the button is, we need to know the row
  To this we set the rows HIGH and ground them 1 pin at a time while reading the result
  on the column pins.
*/
                    KEY_PORT = 0xEF;    // Check for pressed key in 4th row
                    asm("NOP");
                    input = (KEY_PIN & 0x0F);
                    if(input != 0x0F)
                      {
                              rowNum = 3;
                              break;
                    }
                    KEY_PORT = 0xDF;    // Check for pressed key in 3rd row
                    asm("NOP");
                    input = (KEY_PIN & 0x0F);
                    if(input != 0x0F)
                      {
                              rowNum = 2;
                              break;
                    }

                    KEY_PORT = 0xBF;    // Check for pressed key in 2nd row
                    asm("NOP");
                    input = (KEY_PIN & 0x0F);
                    if(input != 0x0F)
                      {
```

```
                        rowNum = 1;
                        break;
                }

                KEY_PORT = 0x7F;    // Check for pressed key in 1st row
                asm("NOP");
                input = (KEY_PIN & 0x0F);
                if(input != 0x0F)
                  {
                        rowNum = 0;
                        break;
                }
        }

/*After obtaining the column and row values, the final keypad key can be output by cross
referencing
  the rows and columns in our keypad character array
*/
        if(colNum == 0x0E)
        return(keypad[rowNum][3]);
        else if(colNum == 0x0D)
        return(keypad[rowNum][2]);
        else if(colNum == 0x0B)
        return(keypad[rowNum][1]);
        else
        return(keypad[rowNum][0]);
}
```

Individual Exercise (20 marks)

Introduction:

While working with most of electronic projects, the keypad is a common component and for a 4×4 keypad with 16 keys and it requires 8 digital I/O lines of the microcontroller device while interfacing. GPIO pins are valuable assets and we need to reserve them as much as we can. There is a way of interfacing the 4x4 keypad using a single analogue pin which enables one to save a considerable amount of the microcontroller pins for other uses.

1. Research on the implementation of this kind of 4x4 interfacing using 1 wire.
2. Create a Proteus circuit which has an ATmega32 microcontroller, a 16x2 LCD and a 4x4 keypad connected to only Pin PA0. (5 mks)
3. Write a suitable C code in Atmel studio to run the device. (10 mks: awarded during presentation)
4. Write a 3 page report (excluding the title page and references page), highlighting the working principle, connecting circuit, working code and pros and cons of this kind of keypad interfacing. (5 mks)

- *You will demonstrate and explain the working program to the lecture with a 5 minutes question and answer session. Marks will be awarded for a compact program, clear explanation and deep understanding of the concept.*
- *This exercise is part of the Continuous Assessment of this course.*
- *Failure to submit on the appointed time will result in a zero score for the exercise.*

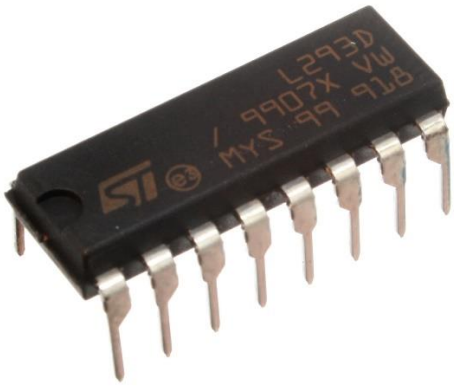
Lesson 3:

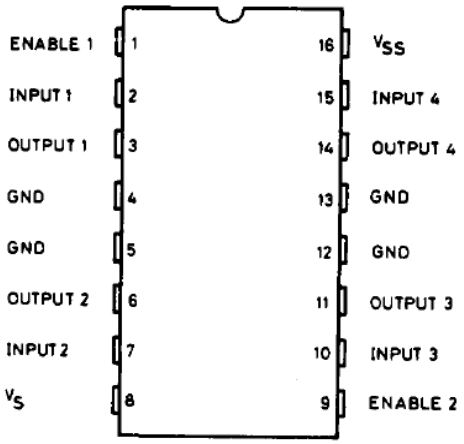
Motor Interfacing:

One of the main applications of an embedded controller is to introduce control into physical systems. In order to do this, it has to interact with the physical world through sensors and actuators. Sensors, enable the control system to obtain data and actuators enable the system to interact and change the physical world. One of the most common form of actuators is the motor. Motors come in various shapes, sizes and with different characteristics but in this course we will focus on 3 main types i.e. Direct current (DC) motors, Servo motors and Stepper motors. Motors in general convert Electrical energy to mechanical energy through electromagnetic interactions. This course will not go too much into the details of motor design and operation but instead we will focus on how to interface these motors with microcontroller systems.

3.1: DC Motor Interfacing.

As mentioned a DC motor converts electrical energy in the form of Direct current into mechanical energy in the form of rotational movement of the motor shaft. The motor can be rotated at a certain speed by applying a fixed voltage to it. If the voltage varies, the speed of the motor varies. Thus, the DC motor speed can be controlled by applying varying DC voltage; whereas the direction of rotation of motor can be changed by reversing the direction of current through it. For applying varying voltage, we can make use of PWM technique. For reversing the current, we can make use of H-Bridge circuit or motor driver ICs that employ the H-Bridge technique or other any other mechanisms. One of the more common IC used for this purpose is the L293D.





L293D is a 16 pin motor driver IC consist of quadruple half H drivers. It can simultaneously control the direction and speed of two DC motors. The IC has an operating voltage range from 4.5 V to 36 V. The L293 and L293D models can drive current up to 1A and 600mA respectively.

IC L293D pin functions

Below is a brief description of the L293D pin operations:

Pin 1, 9: Enable – This is an active high input. When the pin is high it enables the driver channels 1 and 2. Logic HIGH (5V) – Enabled. Logic LOW (0V) – Disabled.

Pin 2, 7, 10 and 15: Input – This controls the output of the output pins. The state of all outputs OUT1, OUT2, OUT3, OUT4 will be same as the input state applied at the corresponding inputs. Output = Input. High input – High output. Low input – Low output.

Pin 3, 6, 11 and 14: Output – Connected to one of the terminals of the motor 1; motor 1 – connected across the output 1 and 2.

Pin 4, 5, 12 and 13: GND – Heatsink and Ground Connection. The GND connection itself used as the heat sink to disperse the heat.

Pin 8: Vcc2 – Supply to the motors, 4.5V to 36V. The supply must be connected to a source capable enough to drive the current requirement of the load.

Pin 16: Vcc1 – 5V supply for the functioning of the IC.

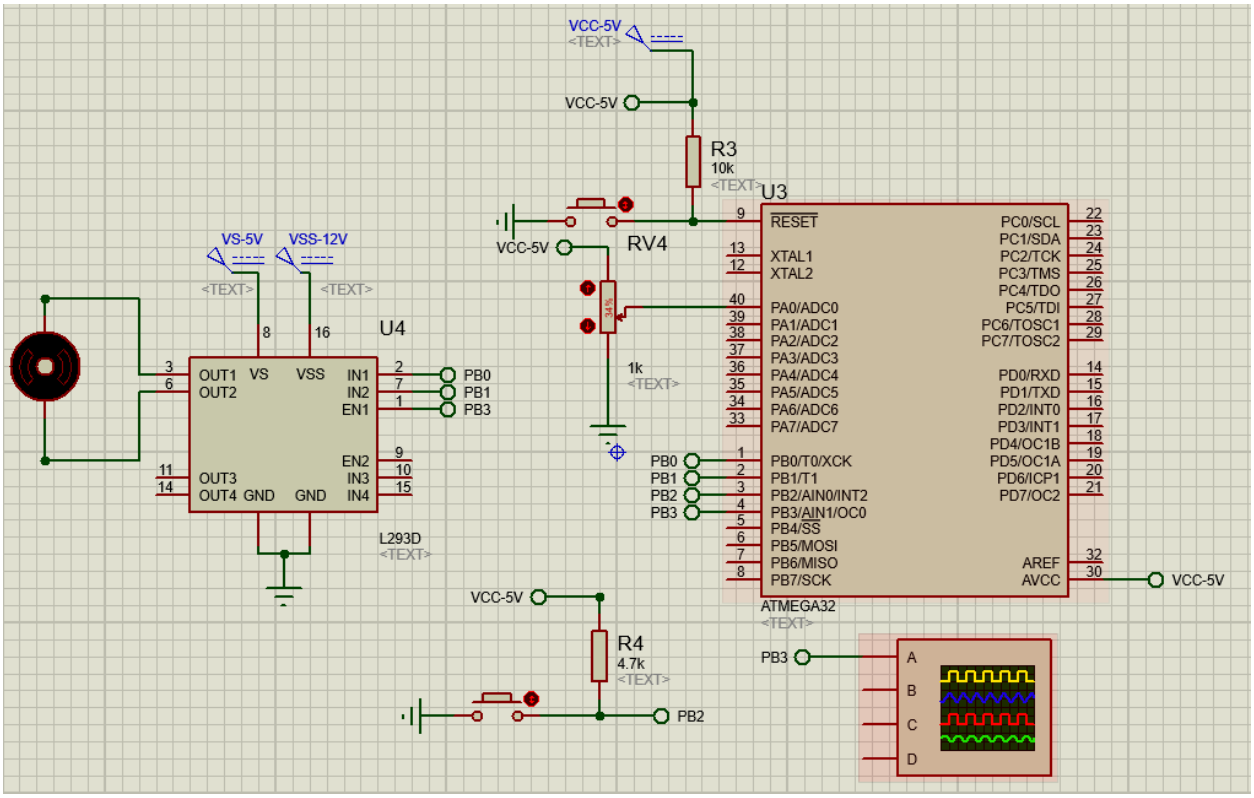
Simply, what a motor driver does is it act as a current amplifier which gives high current outputs to drive the motor from a low current control signal. Driver IC or a driver circuit is a similar H bridge arrangement instead of switches replaced with transistors, MOSFETs, etc. Hence low current input signals can switch these devices and operate in the same way as an H bridge circuit works.

L293D Function Table

INPUT		OUTPUT		Motor Direction
IN1/IN3	IN2/IN4	OUT1/OUT3	OUT2/OUT4	
0	0	0	0	Brake - inactive
1	0	1	0	Clockwise
0	1	0	1	Anti-Clockwise
1	1	1	1	Brake – active(Hold)

Thus from the function table we can see if we want to drive the motor clockwise, we need just to set the IN1 – HIGH and IN2 – LOW. We can invert these signals and the system will run anti-clockwise. If we want to stop current to the motor we set both pins LOW, but if we want to actively brake the system and hold it at a given position, we set both pins HIGH. With this understanding we can now look at the circuit and code implementation for a simple motor speed control system.

Circuit Implementation:



The circuit implementation is as shown below. The L293D is connected to the motor at OUT1 and OUT2, while IN1, IN2 and EN1 are connected to PB0, PB1 and PB3. The enable pin (EN1) is connected to the ***PWM*** pin PB3 since speed control is done through varying the voltage to the enable pin through PWM. PB2 ***interrupt*** pin is connected to a button which is then used for direction change i.e. clockwise to anti-clockwise. PA0 is connected to a potentiometer and the ***ADC*** feature is used to read in the voltage value from the potentiometer. Using this value we can change the PWM output of PB3 and thus vary the speed.

Code Implementation:

```
/*ADC_Functions.c
 * Author : Michael Mureithi
/*Typical Header file inclusion and CPU frequency definition*/
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>


void ADC_Init()                                  /* ADC Initialization function */
{
          DDRA = 0x00;                           /* Make ADC port as input */
          ADCSRA = 0x87;                         /* Enable ADC, with freq/128 */
          ADMUX = 0x40;                          /* Vref: Avcc, ADC channel: 0 */
}

int ADC_Read(char channel)              /* ADC Read function */
{
          ADMUX = 0x40 | (channel & 0x07);/* set input channel to read */
          ADCSRA |= (1<<ADSC);                   /* Start ADC conversion */

          while (!(ADCSRA & (1<<ADIF)));         /* Wait until end of conversion */
          ADCSRA |= (1<<ADIF);                   /* Clear interrupt flag */
          _delay_ms(1);                          /* Wait a little bit */
          return ADCW;                           /* Return ADC word */
}
```

In line with our previous project, we would like to write and save the ADC functions in another C file and call the functions in our main program. The ADC functions were discussed in depth in MSP-I. You can refer to them if more explanation is need.

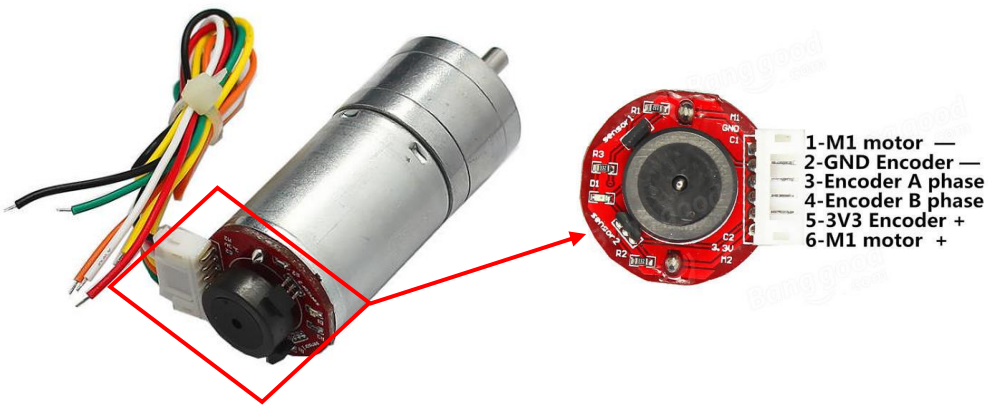
Next we then prepare the header file which contains the function declarations so that the main function can access the functions the ADC function file.

```
/*
 * DC_motor_code.h
 *
 * Created: 17-Sep-19 2:13:53 PM
 *  Author: Michael Mureithi
 */


#ifndef DC_MOTOR_CODE_H_
#define DC_MOTOR_CODE_H_

void ADC_Init();                        /* ADC Initialization function */
int ADC_Read(char channel);             /* ADC Read function */
#endif /* DC_MOTOR_CODE_H_ */
```

Next we define our main c function as shown below

```
/*
 * DC_motor_code.c
 * Author : Michael Mureithi
 */
#define F_CPU 8000000UL                         /* Define CPU Frequency 8MHz */
#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdio.h>
#include <util/delay.h>
#include "DC_motor_code.h"
volatile uint8_t Direction = 0;
/* Function that does the pin definitions and setup for GPI0, interrupt and PWM */
void setup(){
          DDRB = 0xFF;                  /* Make PORTB as output Port */
          DDRB &= ~(1<<PB2);            /* Make INT2 pin as Input */
          DDRB |= (1<<PB3);             /* Make OC0 pin as Output */
          GICR = (1<<INT2);             /* Enable INT2*/
          MCUCSR = (1<<ISC2);           /* Trigger INT2 on Rising Edge triggered */
          sei();                        /* Enable Global Interrupt */
          ADC_Init();                   /* Initialize ADC */
          TCNT0 = 0;                    /* Set timer0 count zero */
          TCCR0 = (1<<WGM00)|(1<<WGM01)|(1<<COM01)|(1<<CS00)|(1<<CS01);/* Set Fast PWM with
Fosc/64 Timer0 clock */
}
/* Interrupt ISR function */
ISR(INT2_vect){
          Direction = ~Direction;               /* Toggle Direction */
          _delay_ms(50);                        /* Software de-bouncing control delay */
}
int main(void)
{
          setup();
          while(1)
          {
                    if (Direction !=0)  /* Rotate DC motor Clockwise */
                    PORTB = 0x01;
                    else                        /* Else rotate DC motor Anticlockwise */
                    PORTB = 0x02;
                    OCR0 = (ADC_Read(0)/4);     /* Read ADC and map it into 0-255 to write in
OCR0 register */
          }
}
```

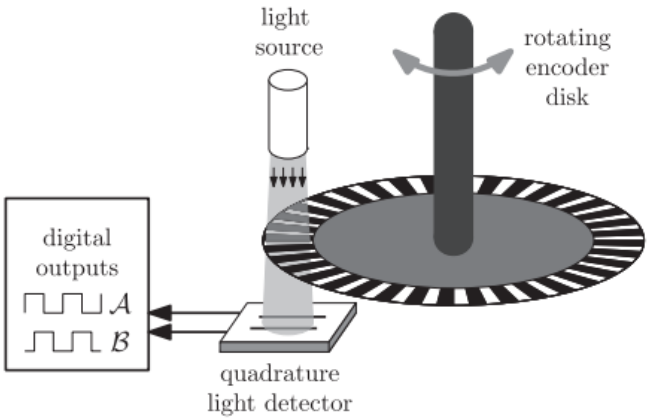
3.2: Encoded DC Motors



Geared encoded motor

Above is an image of a geared encoded motor. The front section of the motor is the gear box which is usually used to enhance the torque. Next is the actual DC motor and at the end the motor encoder is attached. The encoder has 6 pins, 2 for motor power supply i.e. power and ground, 2 for encoder power supply, i.e. encoder power and ground and 2 for encoder output.

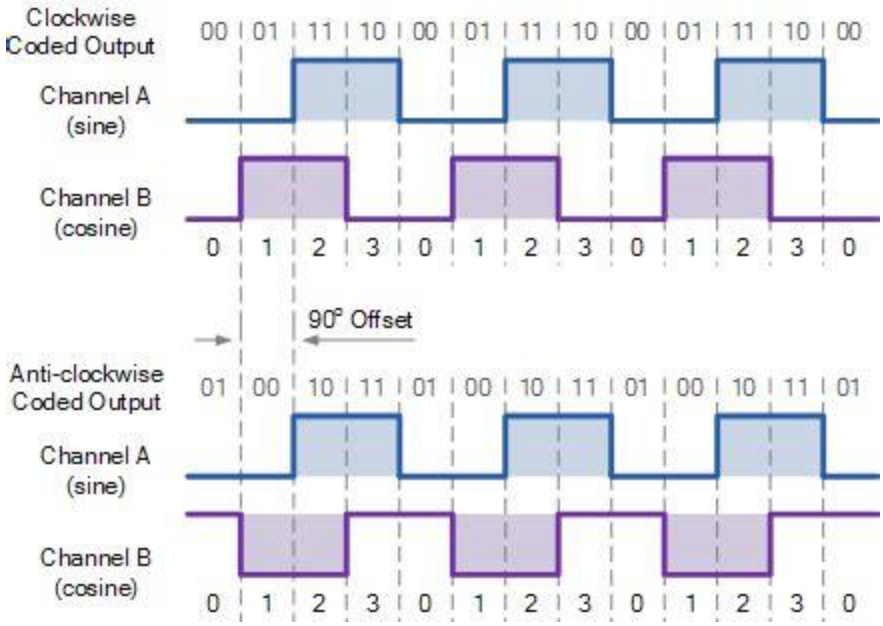
Encoder Operation:



There are various type of encoders. Some measure position and others velocity, while others operate through optical means or through electromagnetic sensing (Hall Effect sensors). We will look at optical velocity encoders since they are quite common in embedded system application.

Optical Encoders are widely used to measure the speed of DC Motor. Incremental encoders consist of three basic components: a slotted disc, a light source and a dual light detector, as shown in the figure above. The light source shines on the disc, which has a regularly spaced radial pattern of transmissive and reflective elements, called encoder increments. The quadrature light detector measures the amount of light passed through the slotted disc and generates two quadrature output pulse signals, denoted by A and B. The up and down changes of the pulse signals are counted as a measurement of the encoder position.

For the application of feedback control to motion systems with optical incremental encoders, the position is generally measured at a fixed sampling frequency. To improve velocity information obtained from optical incremental encoders; fixed time methods are used, which measures the travelled encoder counts over a fixed period-time.



An encoder with one set of pulses is sometimes not sufficient because it cannot indicate the direction of rotation. Using two code tracks with sectors positioned 90 degree out of phase (as shown in Figure 2); the two output channels of the quadrature encoder indicate both position and direction of rotation. For example, if A leads B, the disk is rotating in a clockwise direction. If B leads A, the disk is rotating in a counter-clockwise direction. *Therefore, by monitoring both number of pulses and the relative phase of signals A and B, the microcontroller can track both the position and direction of rotation.*

In addition, some quadrature encoders include a third output channel – called a zero or reference signal – which supplies a single pulse per revolution. This single pulse can be used for precise determination of a reference position. This signal is called the Z-Terminal or the index in most of encoder.

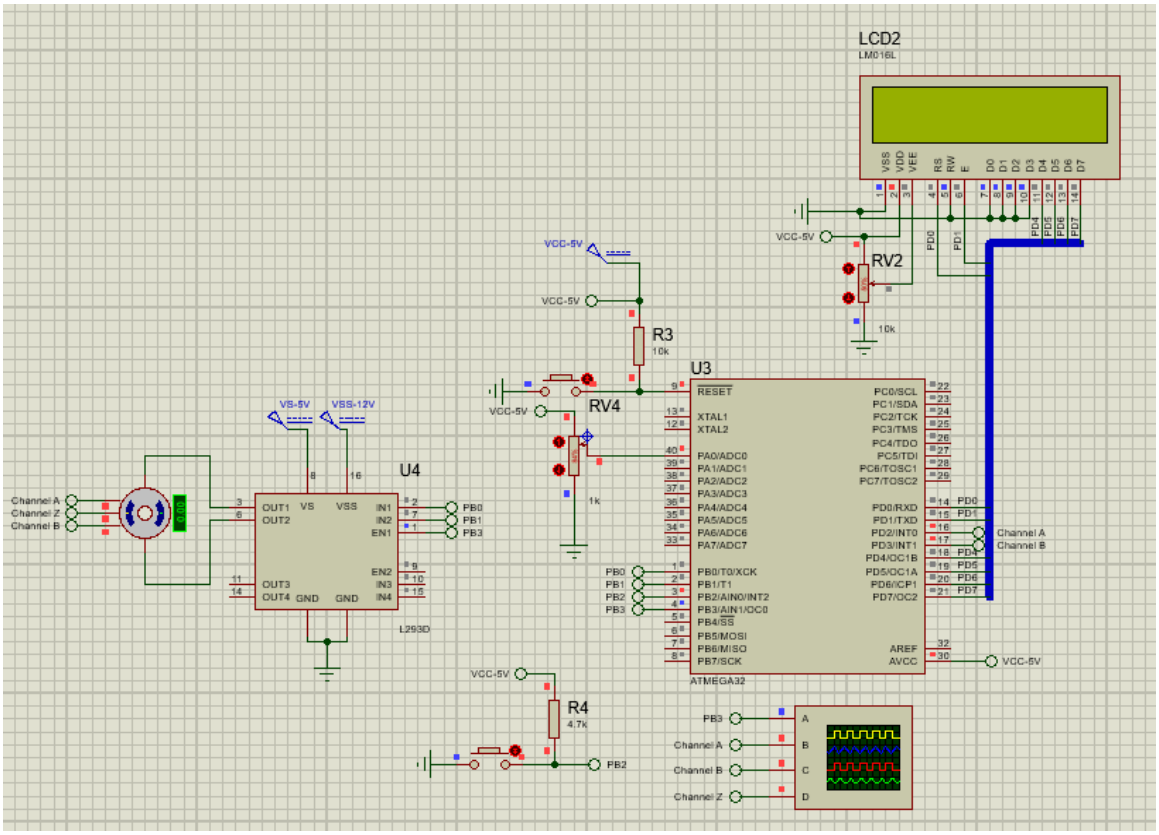
To determine the velocity from the A and B channels, a time period can be set and the number of pulses that are triggered within the time period can help determine the RPM of the motor. In this example, a time period of 500ms was used. Timer1 was used to generate this time period and when the timer overflowed, the current count and previous count were compared.

$$Motor\ Velocity(RPM) = \frac{\text{Change in counts}}{Change\ in\ time} = \frac{Current\ counts - Previous\ counts}{time\ period}$$

This formula will be used to determine the velocity of the motor.

Circuit Implementation

The circuit implementation is a shown in the figure below. It consists of an encoded motor where the A channel and B channel are connected to the interrupt pins INT0 and INT1. We will use interrupts to count the pulses and help us know whether the motor is spinning clockwise or anticlockwise. An LCD is included to view the number of counts and the velocity of the motor shaft.



Code Implementation:

Next we look at the functions that we will use to read the interrupt pins from the encoder channels. In this circuit, we will use INT0 and INT1 for channel A and B plus INT2 is connected to a button switch used to switch the motor rotation direction. We use a function to setup the interrupts and next write the interrupt service routine for each interrupt.

```
/*Interrupt setup function*/
void interrupt_setup(){
        DDRB = 0xFF;                    // Make PORTB as output Port
        DDRB &= ~(1<<PB2); // Make INT2 pin as Input
        DDRB |= (1<<PB3);  // Make OC0 pin as Output
        DDRD &= ~(1<<PD2); // Make INT0 pin as Input
        DDRD &= ~(1<<PD3); // Make INT0 pin as Input
        GICR = (1<<INT2)|(1<<INT1)|(1<<INT0); // Enable INT0, INT2
        MCUCR = (1<<ISC00)|(1<<ISC10);        // Trigger INT0 on Logic change trigger
        MCUCSR = (1<<ISC2);// Trigger INT2 on Rising Edge triggered
        sei();                          // Enable Global Interrupt */
}
/* Interrupt ISR functions */
ISR(INT0_vect){
        cur_encode = PIND & ((1<<PD2)|(1<<PD3));
        cur_encode = (cur_encode>>2);
        // From the encoder value chart, when Channel A changes logic states we look
        // at the value of the interrupt pins. If they are either 0b 11 or 0b 00,
        // the motor is moving CW and we decrease the count
        if(cur_encode == 0x03 || cur_encode == 0x00){
                count-=1;
        }
        // If they are either 0b 10 or 0b 01, the motor is moving CCW and we increase the count
        else if(cur_encode == 0x02 || cur_encode == 0x01){
                count+=1 ;
        }
}
```

```
ISR(INT1_vect){
        cur_encode = PIND & ((1<<PD2)|(1<<PD3)); // Obtain the reading from the PIND2 and PIND3
        cur_encode = (cur_encode>>2);
        // From the encoder value chart, when Channel B changes logic states we look
        // at the value of the interrupt pins. If they are either 0b 01 or 0b 10,
        // the motor is moving CW and we decrease the count
        if(cur_encode == 0x01 || cur_encode == 0x02){
                count-=1;
        }
        // If they are either 0b 11 or 0b 00, the motor is moving CCW and we increase the count
        else if(cur_encode == 0x03 || cur_encode == 0x00){
                count+=1;
        }
}

ISR(INT2_vect){
        Direction = ~Direction;      // Toggle Direction
        _delay_ms(50);               // Software de-bouncing control delay
}
```

Next we look at setting up timer1 for count sampling. The sampling period is set to 500ms. After timer calculations, with 8 Mhz, and timer1, the prescaler was set at 64 and the counter value at 3036. Thus the timer setup function and timer overflow ISR are as shown below. Every time sample, we apply the velocity formula described earlier to obtain the RPMs.

```
/*Timer setup function*/
void timer_setup(){
        TIMSK |= (1<<TOIE1); // Activate the timer overflow interrupt
        TCCR1B = (1<<CS11)|(1<<CS10); // Set the timer prescalar to 64
        TCNT1 = 3036; // Load the countdown value for 500ms
}

/*Timer overflow ISR*/
ISR(TIMER1_OVF_vect){
        cur_count = count;
        RPM = (cur_count-prev_count)*120/(96); // Calculate the RPMs
        prev_count = cur_count;
        TCNT1 = 3036;
}
```

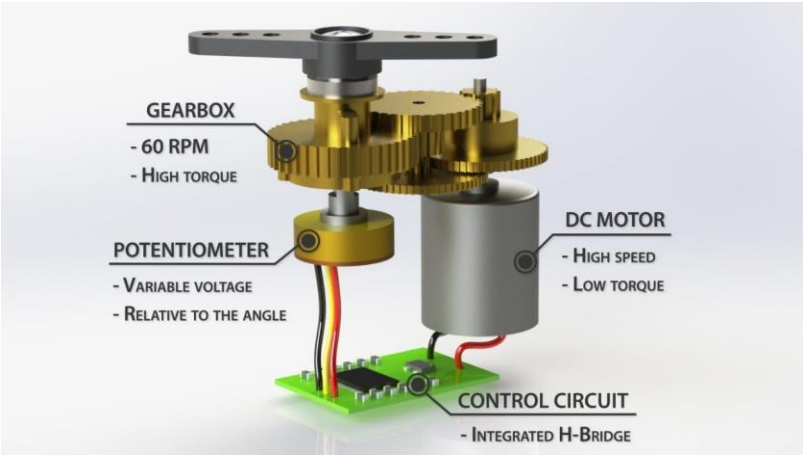
The main function is as shown below, we obtain

```
*Main function*/
int main(void){
        /*Calling all necessary setup/initialization functions*/
        pwm_setup();
        interrupt_setup();
        timer_setup();
        ADC_Init();
        LCD_Init();
        while(1){
                if (Direction !=0) // Rotate DC motor Clockwise
                PORTB = 0x01;
                else               // Else rotate DC motor Anticlockwise
                PORTB = 0x02;
                OCR0 = (ADC Read(0)/4);// Read ADC and map it into 0-255 to write in OCR0
                // Display the number of counts and RPM
                LCD_Cmd(0x80);
                LCD_String("Counts = ");
                itoa(count, buffer, 10);
                LCD_String(buffer);
                LCD_String(" ");
                LCD_Cmd(0xC0);
                LCD_String("RPM = ");
                itoa(RPM, buffer1, 10);
                LCD_String(buffer1);
                LCD_String(" ");

        }
}
```

3.3: Servo Motor Interfacing

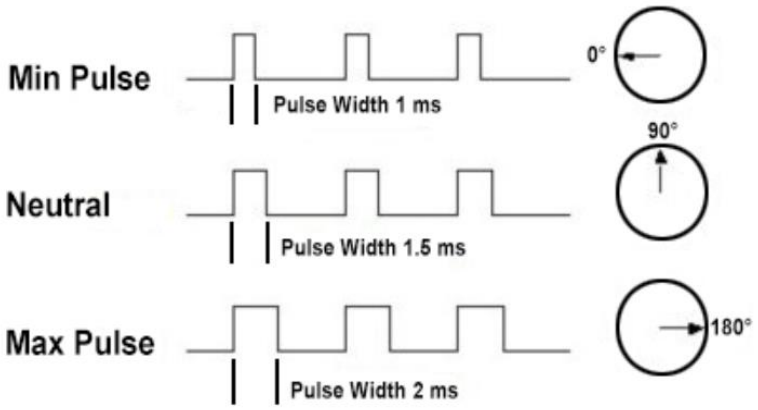
A servo motor is a rotary actuator that allows for precise control of angular position. To fully understand how it works, there is need to take a look at its internal components. Inside there is a pretty simple set-up: a small DC motor, potentiometer, and a control circuit. The motor is attached by gears to the control wheel. As the motor rotates, the potentiometer's resistance changes, so the control circuit can precisely regulate how much movement there is and in which direction.

When the shaft of the motor is at the desired position, power supplied to the motor is stopped. If not, the motor is turned in the appropriate direction. The desired position is sent via electrical pulses through the signal wire. The motor's speed is proportional to the difference between its actual position and desired position. So if the motor is near the desired position, it will turn slowly, otherwise it will turn fast. This is called proportional control. This means the motor will only run as hard as necessary to accomplish the task at hand.



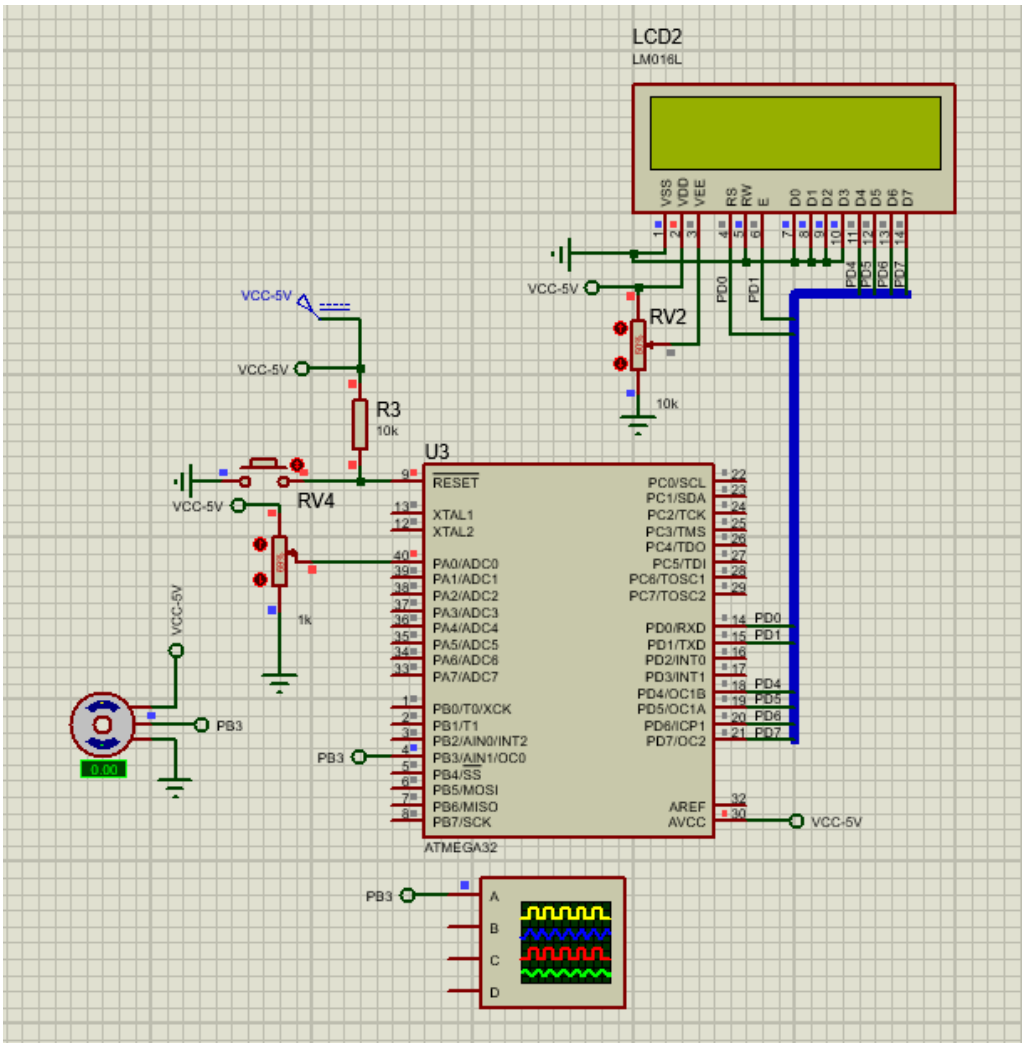
Servo Control Method:

Servo motors are controlled through the signal wire by supplying it with a PWM signal. The width of the PWM signal is the significant factor when controlling the position of the servo motor. For each servo motor there is a minimum and maximum ON time period which corresponds to the minimum and maximum angle the servo motor can reach. For example an ON time of 1ms can correspond to 0 degrees and a maximum ON time of 2ms corresponds to 180 degrees; with all other positions falling between 1 to 2ms ON time.



Circuit Implementation:

The circuit implementation consists of a servo motor whose signal wire is connected to a PWM pin of the microcontroller. The servo motor has minimum ON time value of 1ms and a maximum ON time value of 2ms. A potentiometer is also connected to Pin PA0 and is used to vary the position of the servo motor from its min (-90 degrees) to its max (90 degrees). Thus there is need to map the potentiometer ADC value from 0~1024 to the required 1ms to 2ms PWM ON time values.



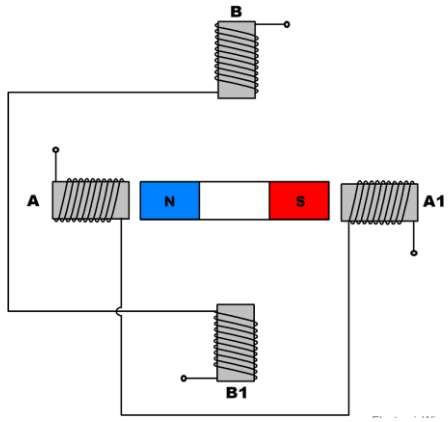

```
/*
 * servo_motor_code.c
 * Author : Michael Mureithi
 */

/* Include files and CPU frequency definition */
#define F_CPU 8000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <stdio.h>
#include <stdlib.h>
#include <util/delay.h>
#include "Servo_code.h"

/* Computation variables */
volatile uint16_t position = 0;
int PWM;
char buffer[20];
char buffer1[20];
```

```
/* Function that does the pin definitions and setup for GPI0 and PWM */
void setup(){
        /* GPIO setup */
        DDRB |= (1<<PB3);  // Make OC0 pin as Output
        /* PWM setup */
        TCCR0 = (1<<WGM00)|(1<<WGM01)|(1<<COM01)|(1<<CS02); // Set Fast PWM with Fosc/256
Timer0 clock
}

int main(void){
        setup();            // Initialize PWM
        ADC_Init();         // Initialize ADC
        LCD_Init();         // Initialize LCD
        while(1){
                position = (ADC_Read(0)); // Obtain ADC reading
                PWM = 31.25 + (position*31.25)/1024; // Scale the ADC value to the correct PWM
values
                OCR0 = PWM;         // Send the PWM value to control the ON time

                /* Displaying the ADC value and the PWM  */
                LCD_Cmd(0x80);
                itoa(position, buffer, 10);
                LCD_String("ADC Value= ");
                LCD_String(buffer);
                LCD_String(" ");

                LCD_Cmd(0xC0);
                itoa(PWM, buffer1, 10);
                LCD_String("Servo Value= ");
                LCD_String(buffer1);
                LCD_String(" ");
        }
}
```

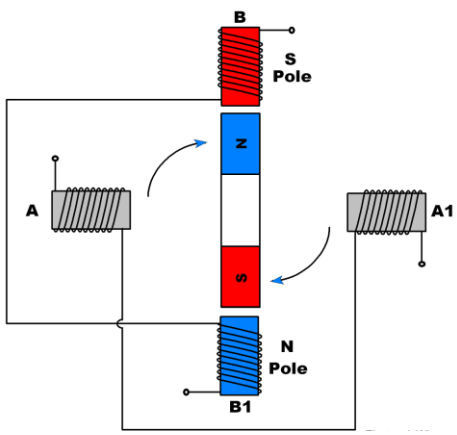
3.4: Stepper Motor Interfacing

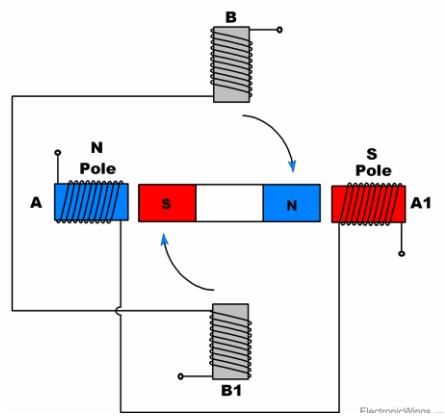
A stepper motor is a brushless DC motor that divides the full rotation angle of 360° into number of equal steps. The motor is rotated by applying certain sequence of control signals. The speed of rotation can be changed by changing the rate at which the control signals are applied. Various stepper motors with different step angles and torque ratings are available in the market. Microcontroller can be used to apply different control signals to the motor to make it rotate according to the need of application.

Working Principle of Stepper Motor:



Current flowing through winding creates magnetic North and South pole on coil. If we wrap our right-hand fingers around coil in the direction of current flowing through the coil, then the thumb direction indicates the magnetic North Pole. Stepper motor rotates in steps. To understand its principle, consider the logical diagram of its construction given below. Two winding, A and B are the stator of motor. Permanent magnet having North and South poles is rotor of the motor. The basic arrangement of stator and rotor in stepper motor is shown in figure below.





By energizing winding B, North and South poles are created on winding s B as shown in the figure above. This will attract opposite poles of magnet towards it. This causes the rotor (permanent magnet) to rotate by a step. Similarly for winding A, when energized, the unlike poles are attracted and make a step as shown in the figure above. By alternating these two steps we can achieve rotation motion of the rotor.

The STEP Angle; is the minimum angle that the stepper motor will cover within one movement/step. The number of steps required to complete one rotation depends upon the step angle. E.g. if step angle is 45 degrees then 8 steps are required to complete one rotation. Typical step angle values range from 0.72, 1.8, 3.75, 7.5, 15 etc.

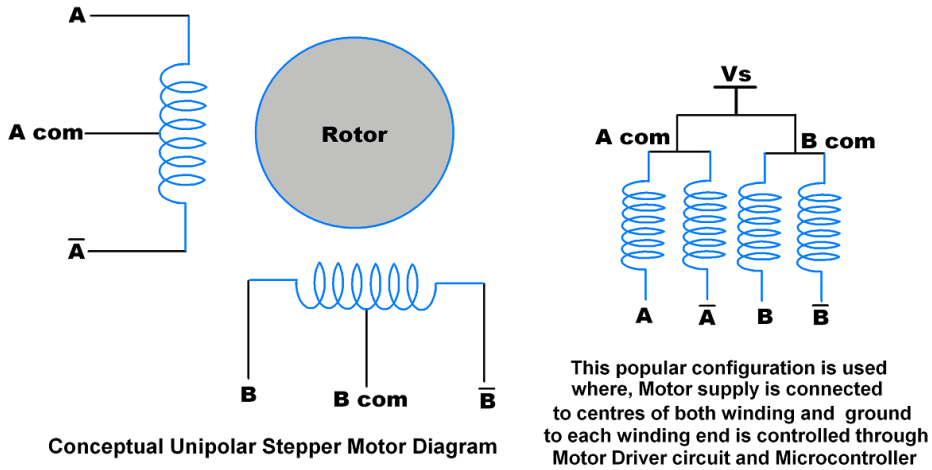
Types of Stepper Motors

Stepper motors are classified depending upon construction and winding arrangements:

1. Depending on winding arrangements:
    a. Unipolar stepper motor
    b. Bipolar stepper motor
2. Depending on construction
    a. Permanent magnet stepper motor
    b. Variable reluctance stepper motor
    c. Hybrid stepper motor.

Unipolar Stepper Motors:

Unipolar stepper motor have centre tapped winding with 5 (If both centres are connected internally) or 6 wires as shown in below figure. Generally, these centre tapped connections are connected to the power supply.

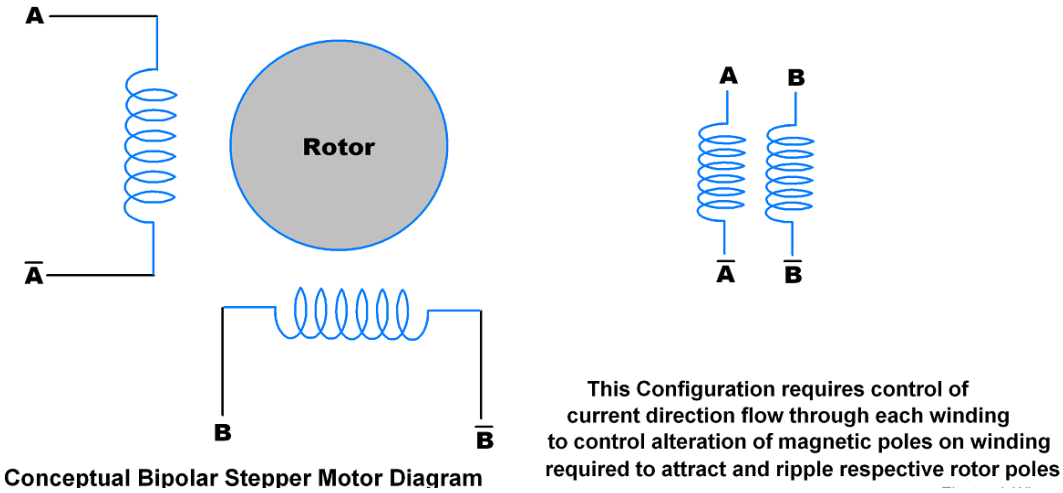


Conceptual Unipolar Stepper Motor Diagram

By providing ground path to the winding leads we can allow the flow of current through each half of coil which create magnetic poles. By altering poles sequentially, we can rotate rotor accordingly. The current is unidirectional in unipolar motor. Due to centre tap, each winding is divided i.e. current flows through half of winding. Due to its centre tap arrangement, we do not need to change current direction to change magnetic pole on winding. Here we just need to alter ground connection at winding ends.

<u>Bipolar Stepper Motor:</u>

Bipolar stepper Motor has no centre tap connection. Normally it is of two windings i.e. 4 wire end as shown in below figure.



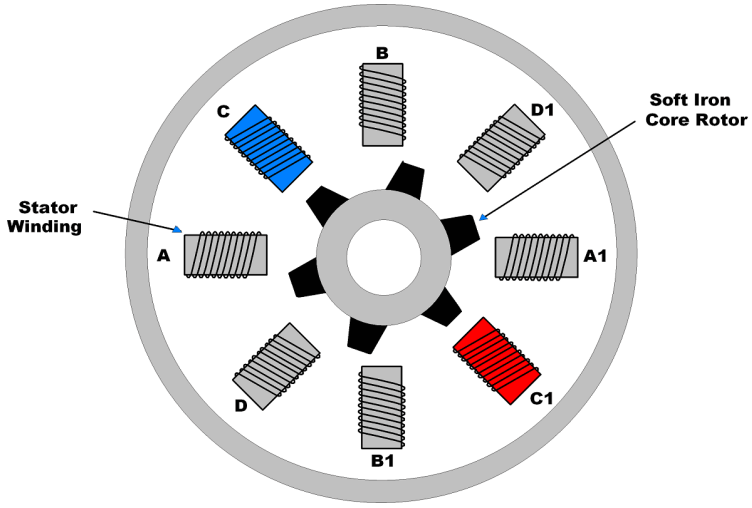
Conceptual Bipolar Stepper Motor Diagram

Current flows through full winding of stator. The current is Bidirectional in Bipolar stepper motor i.e. we need to alter current direction through winding to alter magnetic pole of that winding.

<u>Difference between Unipolar and Bipolar Stepper Motor</u>

1. As per name, they already differ in current direction.
2. Also, due to unidirectional arrangement, Unipolar Stepper Motor do not require to control direction of current. So, it does not require H bridge like circuitry for bidirectional operation. A simple ULN2003 driver circuitry is used to drive it.

3. Whereas in Bipolar Stepper Motor, it requires H bridge driver circuitry e.g. like L293D driver for bidirectional

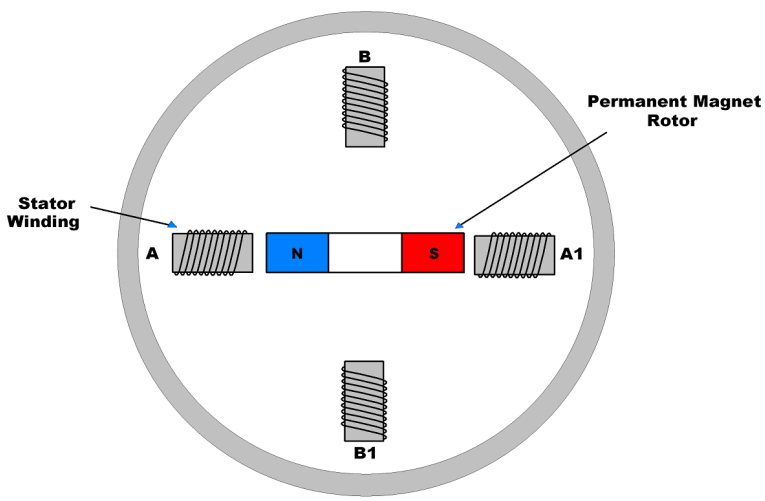


current control.

4. Unipolar Stepper Motor has less torque than Bipolar Stepper Motor because in unipolar stepper motor current flows through half of the winding; whereas in bipolar stepper motor current flows through full winding.

Permanent Magnet Stepper Motor

Here, permanent magnet is used as rotor with electromagnetic stator winding. The rotor and stator of these Stepper Motor are not teethed. Supply is given to the stator, stator winding energized and produce magnetic north and South Pole. This cause the rotor to rotate and to align with energized poles. Now by energizing next winding rotor will step towards it. In this way, continuous sequence of winding pulses causes rotor to rotate continuous.



Variable Reluctance Stepper Motor

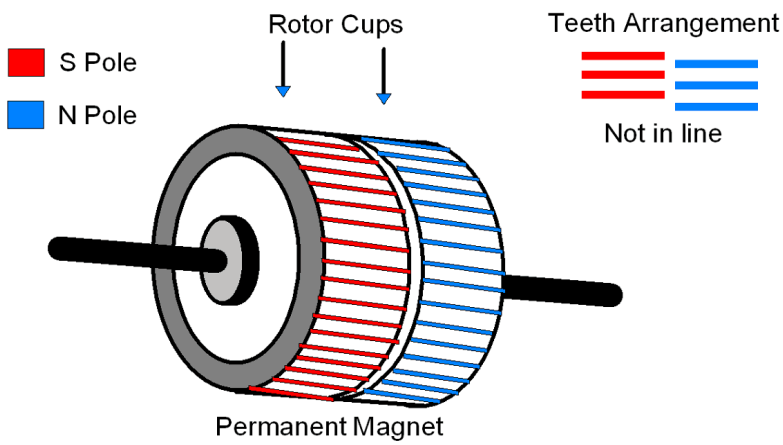
Here Non-magnetic iron core is used as rotor. Stator is electromagnetic winding around rotor. Rotor consists of teeth. These teeth are attracted towards energized winding as magnetic path is generated around coil and rotor. Rotor experiences torque and aligns with energized coil to minimize the flux path. Now if next winding is energized then rotor will move towards it. In this way, continuous sequence of winding pulses cause rotor to rotate continuously.

Note that rotor teeth are arranged in such manner, that at a time only one rotor teeth and energised winding coil pair will align while other teeth slightly deviate with other windings. As shown in figure below, energised winding C align with rotor teeth.

Hybrid Stepper Motor

The Hybrid Motors are combination of permanent magnet and variable reluctance stepper motor. Permanent Magnet used inside the rotor and iron core outside of rotor. Permanent magnet forms the North and South poles on rotor. It has two rotor cups as shown in figure which consist of permanent magnet teeth. Note that their teeth are arranged in zigzag manner.

Hybrid stepper motors have higher torque, and can achieve smaller step size.

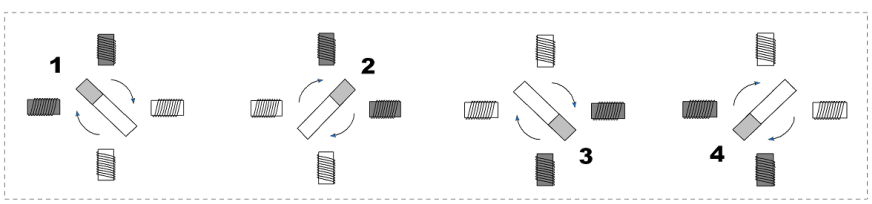


Stepper Motor Control:

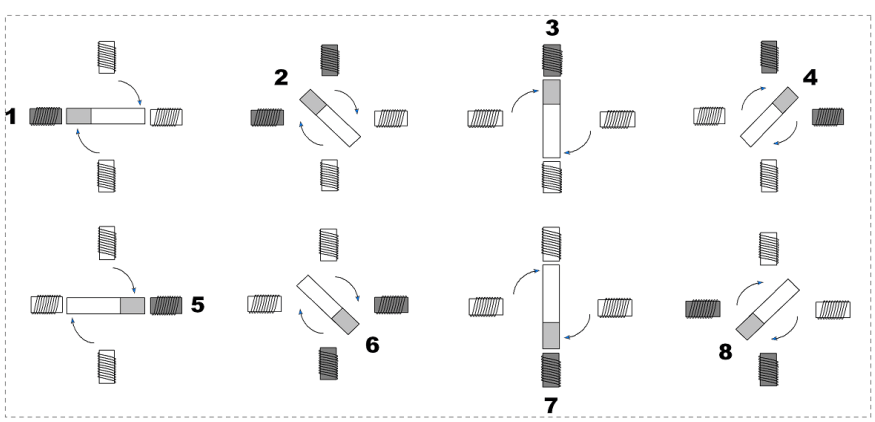
Stepper motor rotates in steps and for continuous or limited angle rotation we need to provide sequential steps. Mostly there are two step sequences used to rotate Stepper Motor as shown in below figure i.e.

Full Step Sequence: Here motor moves through its basic step angle. Two coils are excited at the same time.

Half Step Sequence: Here motor moves half of its basic step angle. Half step can be achieve by exciting both current and next coil.™



Full Step Sequence



Half Step Sequence

This kind of control can be achieved in programming using the step tables below:

Full step sequence

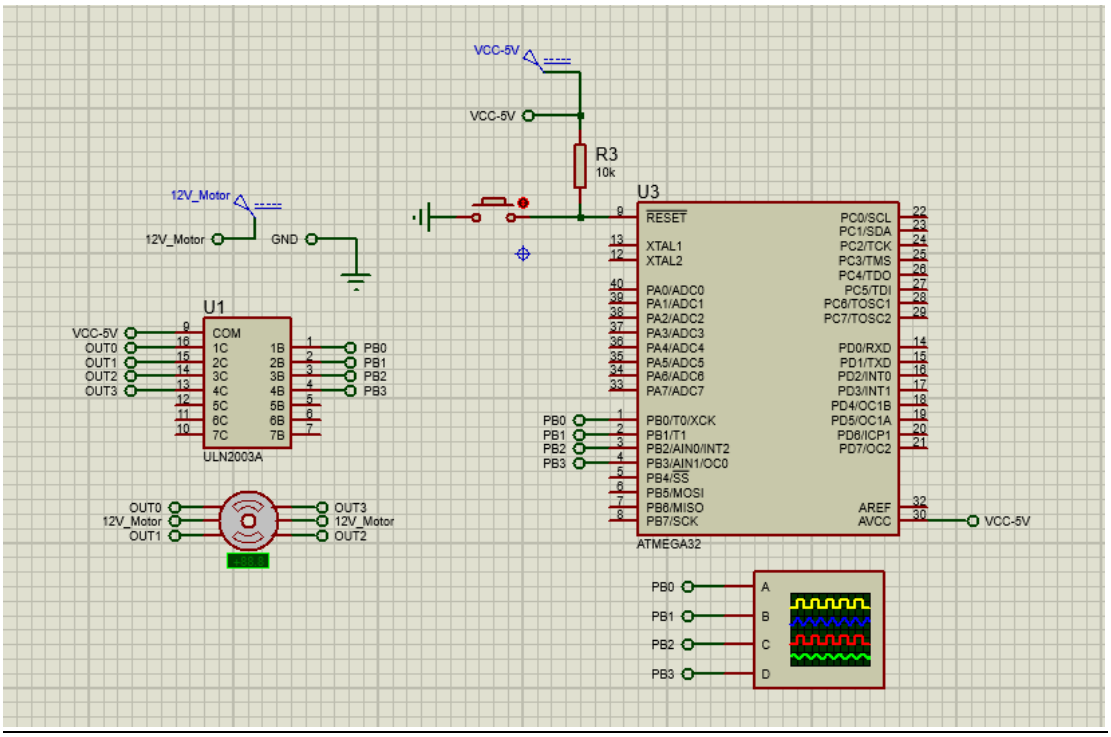
Step	**A**	**B**	**C**	**D**	**Hex**
1	1	0	0	1	0x09
2	1	1	0	0	0x0C
3	0	1	1	0	0x06
4	0	0	1	1	0x03

Half step sequence

Step	**A**	**B**	**C**	**D**	**Hex**
1	1	0	0	1	0x09
2	1	0	0	0	0x08
3	1	1	0	0	0x0C
4	0	1	0	0	0x04
5	0	1	1	0	0x06
6	0	0	1	0	0x02
7	0	0	1	1	0x03
8	0	0	0	1	0x01

Circuit Implementation:

Typical circuit implementation of a Stepper motor control circuit is as shown in the circuit diagram below. The circuit consists of a stepper motor and connected to the microcontroller using the ULN2003A IC. This is an IC containing an array of seven NPN Darlington transistors able to deliver 500mA, at 50V. Motors operate at higher voltages and need to be supplied with higher current than the microcontroller had handle. This is the reason when controlling/interfacing with motors, motor control ICs are regularly used in order to supply the adequate motor voltage and current. More details concerning this IC can be accessed through its data sheet.



Code Section:

Having identified the required motor pin values so as to make the motor rotate in both full and half step, a sequence is created in the "int main" section of the code in order to send the appropriate pin values as shown in the code section below.

```
/*
 * Stepper_Code.c
 */

#define F_CPU 8000000UL          /* Define CPU Frequency 8MHz */
#include <avr/io.h>                      /* Include AVR std. library file */
#include <util/delay.h>          /* Include delay header file */
                                 /* Definition of the motor DDR and PORT */
#define Motor_Dir DDRB
#define Motor_Port PORTB

/* An array containing the Hex values of the stepper motor step values*/
short motor_steps[] = {0x09, 0x08, 0x0C, 0x04, 0x06, 0x02, 0x03, 0x01};

int main(void)
```

```
{
        int period, i = 0;
        Motor_Dir = 0x0F;           /* Motor port initialization */
        period = 100;               /* Time gap between each motor step */

  while (1){
                Motor_Port = motor_steps[i];
                _delay_ms(period);
                i++;
                /* if statement to reset the value of 8 to prevent overflow */
                if(i%8 == 0){
                        i = 0;
                }
        }
        return 0;
}
```

WHAT NEXT?

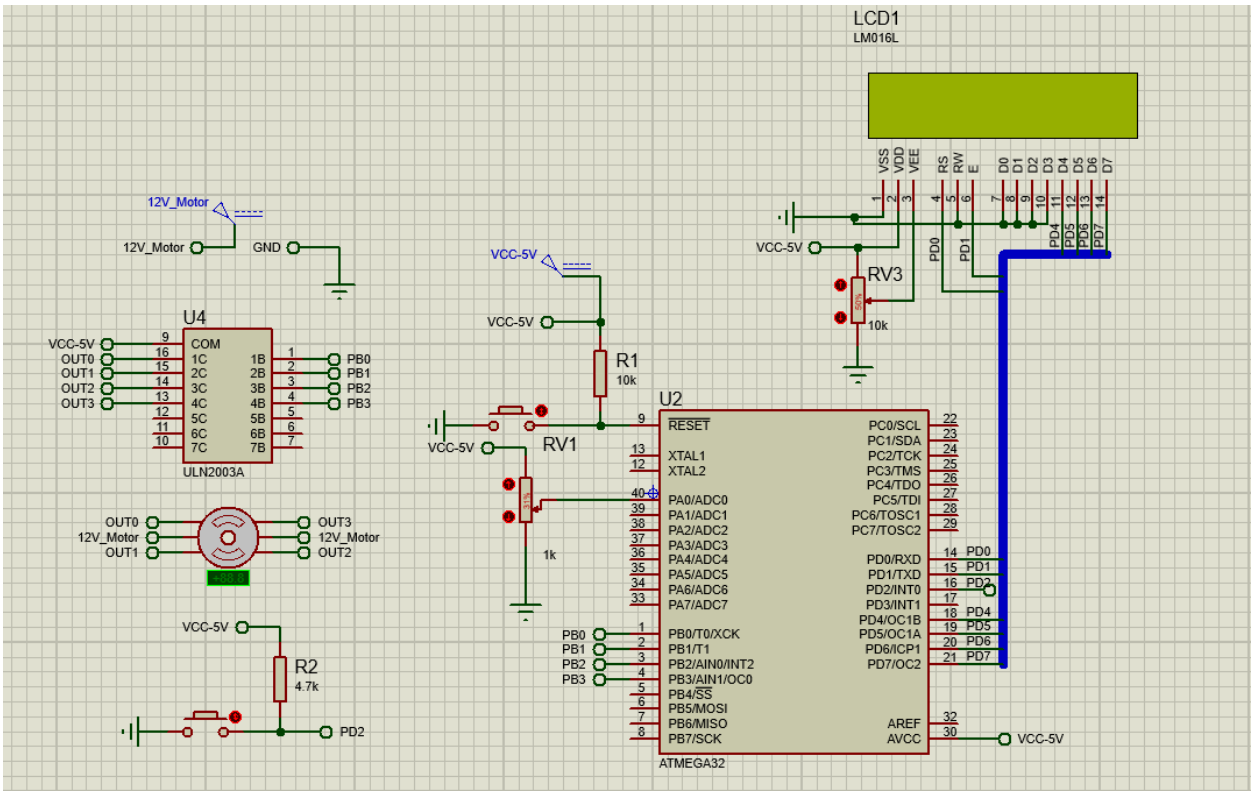
Assignment: 20 mks

Class activity: Using the basics highlighted in the section above, implement an executable C code where you can control the speed of the stepper motor using a potentiometer connected to pin PA0; Maximum speed 150 rpm, Minimum speed 10 rpm. A button connected to PD2, should also be used to change the direction of the motor from clockwise to anticlockwise and vice versa). The speed of the stepper motor in RPM and the motor direction should be displayed on the LCD screen.

Hint:

You will use the Timer, ADC and Interrupt features in the ATmega32 to achieve the desired functionality.

Use the circuit arrangement shown below to implement the above code.


LCD1
LM016L
VSS
VDD
VEE
RS
RW
E
D0
D1
D2
D3
D4
D5
D6
D7
VCC-5V
RV3
10k
PD0
PD1
PD4
PD5
PD6
PD7
12V_Motor
GND
U4
COM
1C
2C
3C
4C
5C
6C
7C
1B
2B
3B
4B
5B
6B
7B
ULN2003A
OUT0
OUT1
OUT2
OUT3
PB0
PB1
PB2
PB3
R1
10k
RV1
1k
U2
RESET
XTAL1
XTAL2
PA0/ADC0
PA1/ADC1
PA2/ADC2
PA3/ADC3
PA4/ADC4
PA5/ADC5
PA6/ADC6
PA7/ADC7
PB0/T0/XCK
PB1/T1
PB2/AIN0/INT2
PB3/AIN1/OC0
PB4/SS
PB5/MOSI
PB6/MISO
PB7/SCK
PC0/SCL
PC1/SDA
PC2/TCK
PC3/TMS
PC4/TDO
PC5/TDI
PC6/TOSC1
PC7/TOSC2
PD0/RXD
PD1/TXD
PD2/INT0
PD3/INT1
PD4/OC1B
PD5/OC1A
PD6/ICP1
PD7/OC2
AREF
AVCC
ATMEGA32
R2
4.7k
PD2